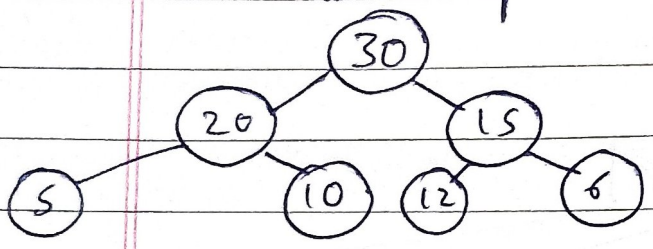
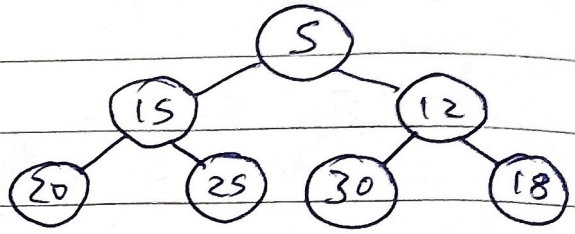
Binary Heap

In simple it's a complete Binary tree

Max Heap



Min Heap



represented mostly using array.

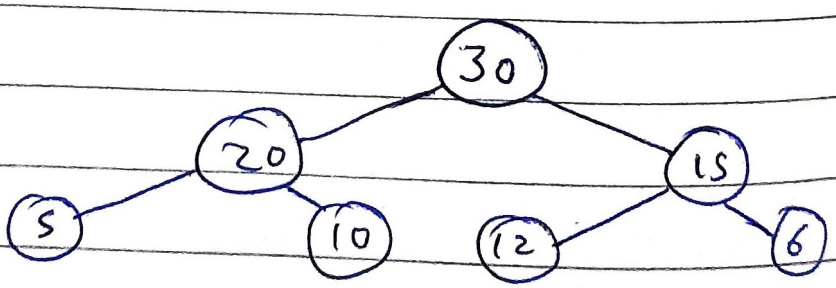
30	20	15	5	10	12	6
1	2	3	4	5	6	7

Conditions

- It's a complete binary tree.
- [Every node should have Ele ≥ decendents
  Every node should have Ele ≤ decendents

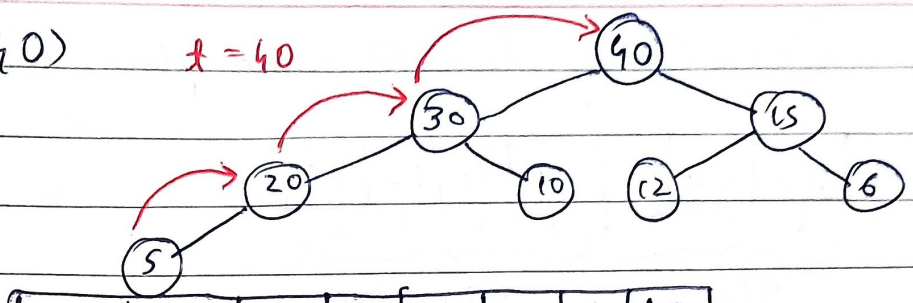
- Hight of Binary Heap is $\log n$
- Not usefull for searching

Insertion



30	20	15	5	10	12	6		
0	1	2	3	4	5	6	7	....

insert (40) $t = 40$

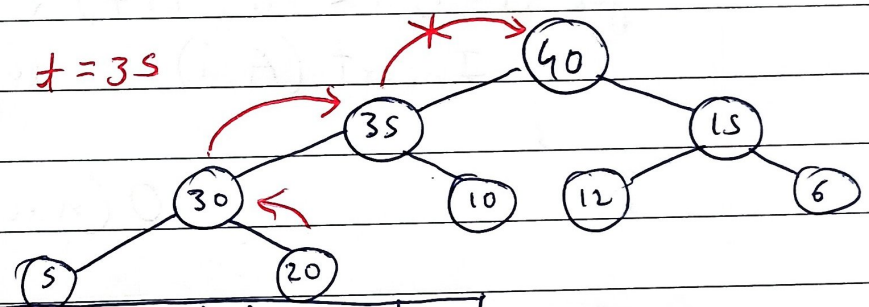


	30	20	15	5	10	12	6	40
before	0	1	2	3	4	5	6	7

Compare if odd $\frac{n-1}{2}$ if even $\frac{n}{2}$

after	40	30	20	15	5	10	12	6

insert (35) $t = 35$



before	40	30	20	15	5	10	12	6
after	40	35	30	20	15	5	10	12	6

Code

```
void Insert (int A[], int n) {
    int temp, i = n;
    temp = A[n];
    while (i > 1 && temp > A[i/2]) {
    A[i] = A[i/2];
     i = i/2;
    } A[i] = temp;
    }
```

$O(\log n)$

Creating Heap

In place Heap creation

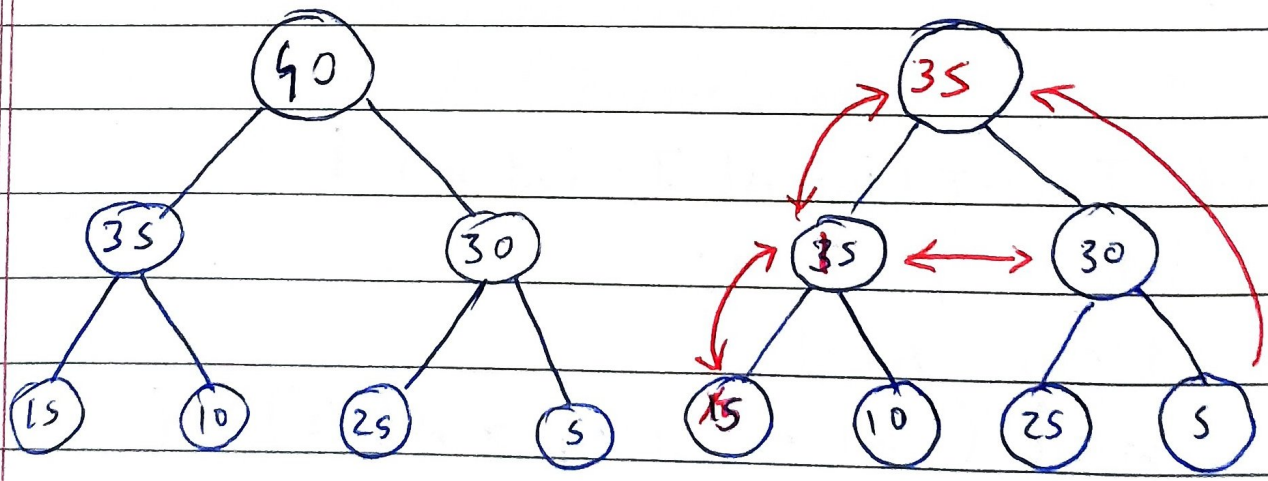
30	20	10	40	15	12	25
0	1	2	3	4	5	6

Heap | not a Heap

```
void createHeap() {
  int A[] = {    };
  int i;
  for (i=1; i<=n; i++) {        n
      Insert(A, i);             logn
    }
}                               O(n logn)
```

Deleting from Heap

Only root can be deleted



Before	40	35	30	15	10	25	5
After	35	15	30	5	10	25	40

```
void Delete (int A[], int n) {
    int x, i, j;
    x = A[n];
    A[i] = A[n];
    i = 1, j = 2 * i;
    while ( j < n-1 ) {
     if (A[j+1] > A[j])
            j = j+1;
     if (A[i] < A[j]) {
            swap (A[i], A[j]);
                i = j;
                j = 2*j;
            } else break;
        } A[n] = x; }
```

Also known as heap sort

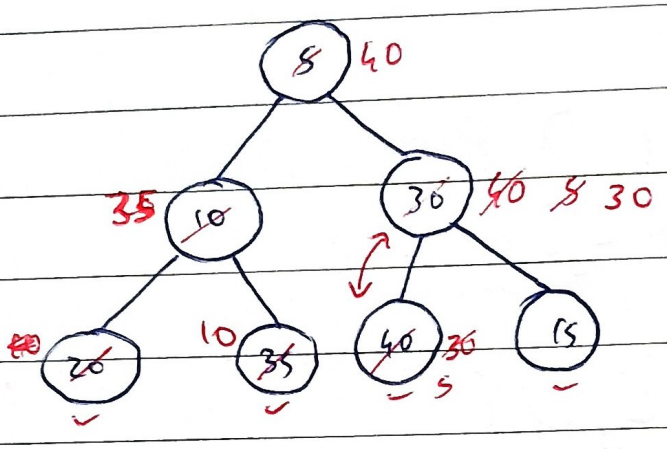
Heap Sort

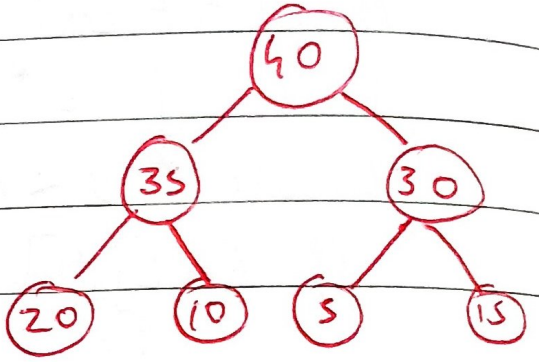
1. Create Heap of n elements $O(n \log n)$
2. Delete 'n' ele one-by-one $O(n \log n)$

$O(2n \log n) = O(n \log n)$

Heapify

5	10	30	20	35	40	15





BH v/s Priority Queue

ele → 4, 9, 5, 10, 6, 20, 8, 15, 2, 18

Larger the number (Doesn't matter)
higher the priority

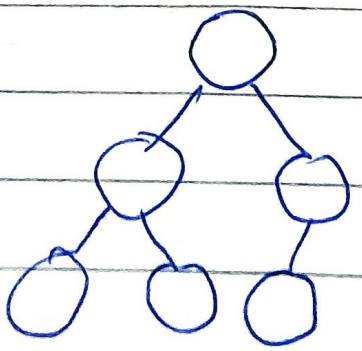
A

4	9	5	10	6	20	8	15	2	18

Insert - O(1)

Delete - O(n) (n + n 2n O(n)

BH -



Insert - O(logn)

Delete - O(logn)